

分支限界

《算法设计与分析》

信息科学与技术学院 曾华琳



01



算 法 思 想



引言(Introduction)

- 与回溯法一样，分支限界是搜索一个解空间，而这个解空间通常组织成一棵树。
- 常见的树结构：子集树和排列树。
- 回溯以深度优先搜索一棵树，而分支限界常常以广度优先或最小耗费优先的方法搜索这棵树。



引言(Introduction)

- 解空间树。
- **分支限界法：**常用于解最优化问题。
 - 确定所求问题的上下界。
 - 在每个节点使用限界函数来屏蔽节点或是扩展节点。
 - 然后，使用目前为止最好的解来帮助剪枝，直到所有顶点被遍历或是被剪掉。

分支限界法

- 假如我们在考虑一个最小化问题时。我们的想法是使所有的可能目标函数值必须维持在上界（目前为止最好的解的目标函数值）与下界之间。如果在某一数量的决策之后（转移操作），我们到达一个节点，在这个节点上我们得到的下界大于或等于上界，那么就没有必要在扩展这个节点既不需在延伸这个分支。
- 对于最大化问题规则正好相反：一旦上界小于或等于先前确定的下界，那么就剪掉这个枝。



分支限界法

- 首先，分支限界是对最优化问题可行解进行剪枝的一个方法。
- 将搜索集中在有希望得到解的分支上。也就是说，在基于上下界和可以得到最优解的基础上，扩展分支，理由是发展这样的分支可以得到更好的上下界，从而可以剪去更多的分支。
- 总之，不要单纯以DFS或BFS来进行搜索，而要结合起来进行搜索。

分支限界法

分支限界需要的步骤如下:

1. 对所求的问题定义一个解空间。这个解空间至少包含这个问题的最优解。
2. 对解空间进行组织，以便能更好的搜索。比较常见组织方式有图或是一棵树。
3. 以广度优先搜索的方式搜索解空间，使用限界函数来避免那些不能得到解的子空间



分支限界法

- 分支限界是有系统的搜索一个解空间的另一个方法。首先在扩展节点的扩展方法上，它不同于回溯。
- 每个活节点变成扩展节点只有一次。当一个节点变成扩展节点时，我们展开从它可到达的所有节点。其中那些不能得到可行解的节点去掉（成为死节点），把剩下的节点加到活节点的表中，然后，从这个表中选一个节点作为下一个扩展节点。

分支限界法

从活节点的表中选一个节点并扩展它。这个扩展操作持续到找到解或这个表为空为止。

选择下一个扩展节点的方法：

1) 先进先出 (FIFO)

这个方法是按节点放进表中的次序从活节点表中选择节点。这个活节点表可被看作一个队列。使用广度优先来搜索这个棵树。

分支限界法

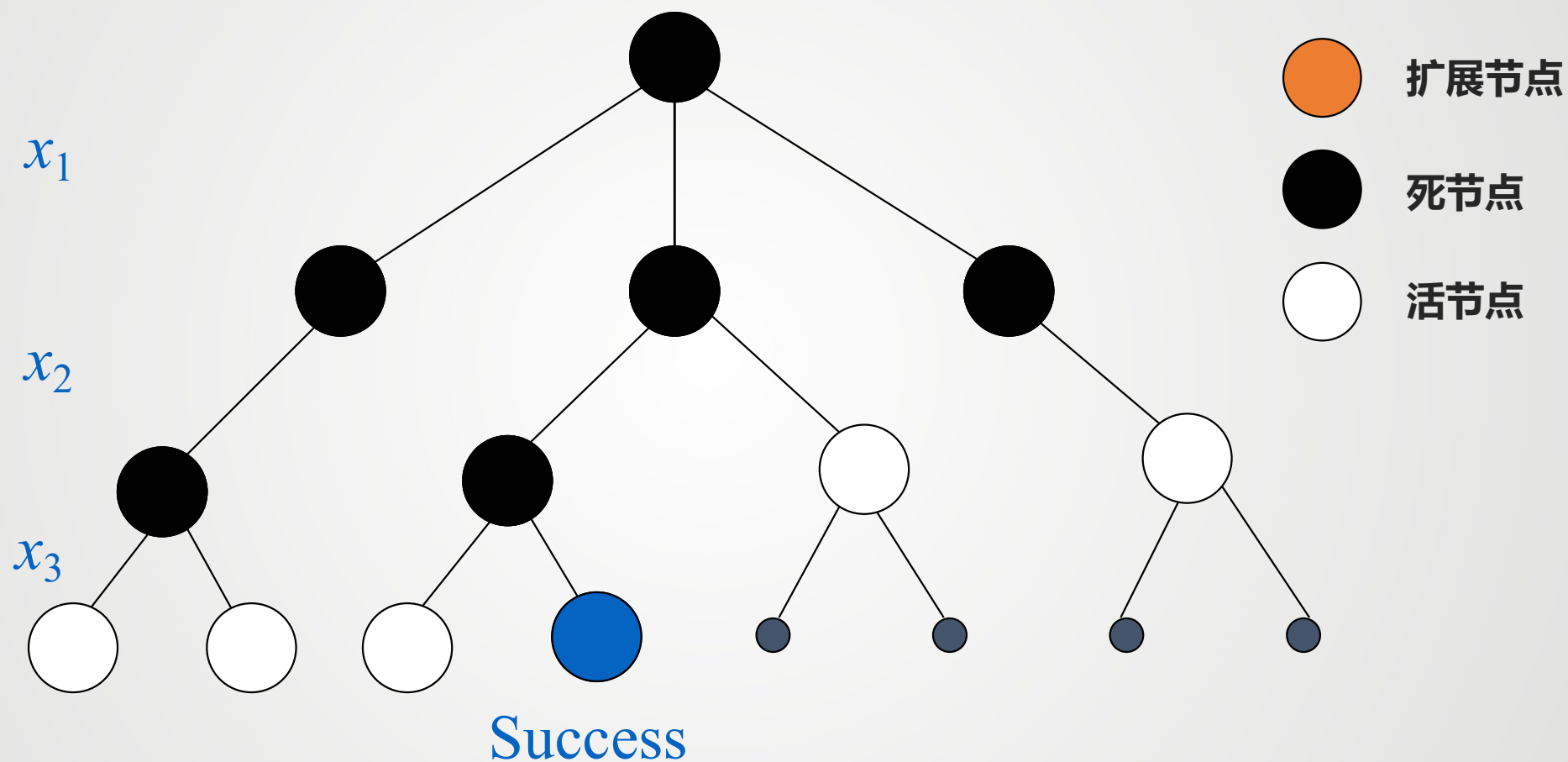
注意分支限界的FIFO不同于广度优先的FIFO在于它不搜索那些不可行的节点。

2) 最小花费，最大收益

用一个优先队列代替一个FIFO队列。这个方法与每个节点的花费或收益相关。如果我们搜索最小花费的解时，那么活节点表就用一个最小堆来表示。下一个活节点是花费最少的节点。如果我们要得到最大收益的解时，活节点表可用一个最大堆来表示。下一个扩展节点为最大收益的节点。



解空间树(Solution space tree)



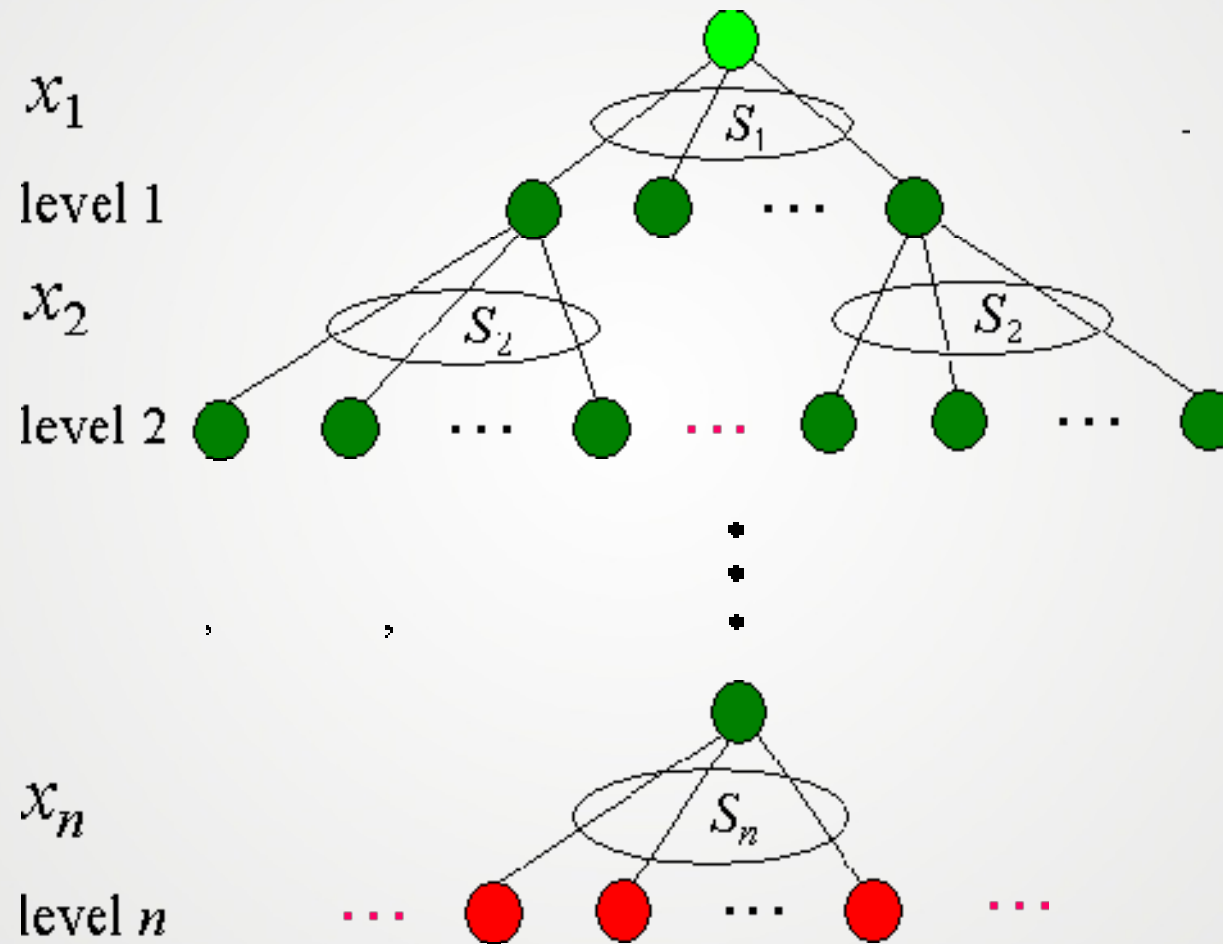
分支限界法

使用分支限界至少需要注意以下4点:

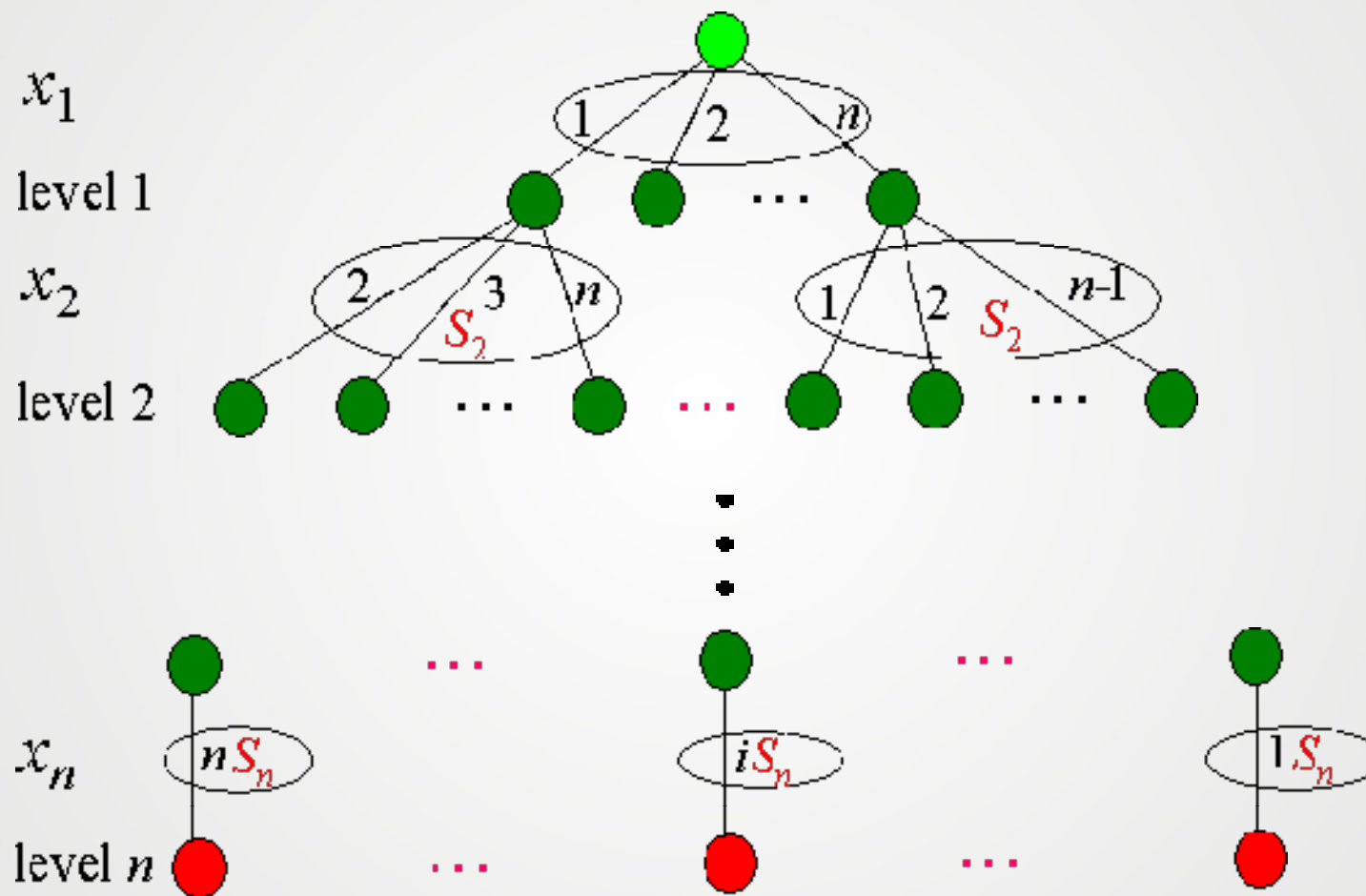
1. 怎么样计算上界 (极大值问题)
2. 怎样计算下界 (极小值问题)
3. 怎样为下一个分支操作选择一个节点
4. 怎样扩展一棵搜索树 (BFS, DFS, ...)

充分利用限界函数和约束函数来剪去无效的枝并把搜索集中
在可以得到解的分支上。

子集树(Subtree)



>>> 排列树(Permutation tree)



A person's hand is visible on the left, holding a silver marker and drawing a circle on a whiteboard. The hand is also holding a document with a colorful diagram consisting of four circles (green, blue, orange, red) arranged in a square. The whiteboard has several faint circles drawn on it. The background is a bright, modern office space with large windows. The text "谢谢大家" is written in large, bold, blue characters on the right side of the whiteboard.

谢谢大家